Assessment 1

SQL and Database Testing

Theme: User Management System

Section A: Conceptual Questions

1. You are testing a web application where user details are stored in a relational database. Explain how understanding database types helps you decide what kind of testing approach is needed.

   - **Verifying Relationships:** Data is stored in multiple linked tables. A tester knows to use SQL Joins to ensure information is correctly connected and displayed.
   - **Validation Rules:** These databases have strict rules (like Unique Email). This allows the tester to create specific cases to check if the system correctly blocks invalid data.
   - **Data Integrity:** It helps the tester verify that updating or deleting a user in one table correctly reflects across all other linked tables, preventing data errors.

2. While validating a search feature, why would filtering data using conditions be more effective than validating random records?

   - **Targeting Edge Cases:** Filtering allows you to search for specific "risky" data, such as users with very long names, special characters, or inactive statuses, which random checking might miss.

   - **Accuracy:** It helps you verify if the search logic actually works. For example, if you filter for "Users from Mumbai," you can quickly see if any "Pune" users incorrectly appear in the results.
   - **Efficiency:** Instead of wasting time checking 100 random healthy records, filtering helps you find and test the exact scenarios that are most likely to fail.

3. During test execution, incorrect user data appears on the user interface. Explain how database validation can help identify the root cause.

   - **Isolating the Bug:** If the data in the Database is correct but the UI shows it wrong, the bug is in the Frontend (UI/Display logic).
   - **Identifying Backend Issues:** If the data in the Database is also wrong, the bug is in the Backend/API (Data processing or saving logic).
   - **Root Cause:** It tells the tester exactly which layer of the application is failing, so the developers know where to fix the code.

4. A tester directly updates user status in the database during testing. Explain when this approach is acceptable and when it is risky.

- **When it is Acceptable:** It is useful for **saving time** and setting up test scenarios. For example, if you need to test a "Account Expired" feature, it is faster to manually change the expiry date in the database rather than waiting for days.
- **When it is Risky:** It is risky because it **bypasses the application logic**. A real user would update their status through the UI, which triggers certain rules and validations. By changing the DB directly, you might create "unnatural data" that doesn't trigger necessary background processes (like sending a notification email).

5. Explain how database constraints reduce the number of defects reaching production

- **Prevents Bad Data:** Constraints automatically reject "invalid" data (e.g., a blank password or a duplicate email) before it is saved, ensuring the system doesn't crash later.
- **Ensures Consistency:** They make sure all data follows the same rules. For example, a "Foreign Key" ensures that if a user is deleted, their old, "ghost" records don't stay in the system to cause errors.
- **Reduces Code Complexity:** Because the database itself enforces these rules, developers don't have to rely only on UI code, which reduces the chance of human error reaching the live production environment.

Section B: Practical SQL Tasks

1. **Table Creation: Create a `Users` table with appropriate data types (e.g., `INT`, `VARCHAR`, `ENUM`). Fields must include: User Identifier (Primary Key), Username, Email, Password, and Status.**

```
CREATE TABLE Users (
   UserID INT PRIMARY KEY,
   Username VARCHAR(50),
   Email VARCHAR(100),
   Password VARCHAR(255),
   Status ENUM('Active', 'Inactive')
   LastLogin Date
);
```

2. **Data Insertion: Write INSERT INTO statements to add 5 test users. Ensure a mix of data for testing, including at least one "Inactive" status and one user with a last login date older than a year**

INSERT INTO `users`(`UserID`, `Username`, `Email`, `Password`, `Status`, `LastLogin`)
VALUES
('1', 'shubham_patil', 'shubham@example.com', 'pass123', 'Active', '2026-04-15'),
('2', 'khimali_p', 'khimali@example.com', 'secure456', 'Active', '2026-04-20'),
('3', 'test_user_old', 'olduser@example.com', 'oldpass789', 'Active', '2024-01-10'),
('4', 'inactive_tester', 'inactive@example.com', 'test321', 'Inactive', '2025-12-01'),
('5', 'admin_user', 'admin@example.com', 'admin777', 'Active', '2026-04-29');

3. **Data Retrieval (10 Queries): Write 10 SELECT queries demonstrating: ● Filtering: Using Where for specific statuses. ● Pattern Matching: Using LIKE (e.g., finding emails ending in @gmail.com). ● Sorting: Using ORDER BY for usernames or identifiers.**

```
SELECT * FROM users WHERE Status = 'Active';

SELECT UserID, Username, Email, Status FROM users WHERE Status = 'Inactive';

SELECT Username, Status, LastLogin FROM users WHERE Status = 'Active' AND LastLogin >= '2026-01-01';

SELECT Username, LastLogin FROM users WHERE LastLogin < '2025-01-01';

SELECT Username, Email FROM users WHERE Email LIKE '%@example.com';

SELECT Username, Email, Status FROM users WHERE Username LIKE 'admin%';

SELECT Username, Status FROM users WHERE Username LIKE '%user%';

SELECT Username, Email FROM users WHERE Email LIKE 'old%' OR Email LIKE 'inactive%';

SELECT UserID, Username, Status FROM users ORDER BY Username ASC;
```

```
SELECT Username, Status, LastLogin FROM users ORDER BY LastLogin DESC;
```

4. **Data Updates: Write an UPDATE statement to change the Status of specific users (e.g., setting "Pending" accounts to "Inactive" based on a condition).**

```
UPDATE users
SET Status = 'Inactive' WHERE
LastLogin < '2025-01-01'
AND Email IS NOT NULL;
```

5. Data Deletion: Write a DELETE query to remove users who haven't logged in for over a year.
● Tip: Use date functions like DATEDIFF () or CURRENT_DATE - INTERVAL '1 year' depending on your SQL dialect

```
DELETE FROM users
 WHERE LastLogin < CURRENT_DATE - INTERVAL 1 YEAR;
```

C: Mini Section Project – End-to-End Database Validation

1. **Test Planning Document: Prepare a Test Plan focusing on database testing scope, risks, test data strategy, and validation approach.**

- **What is this test plan about?**
  This test plan is created for testing the database of a User Management System. The goal is to make sure that all the data stored in the database is correct, complete, and follows all the rules defined during table creation

- **What will we test?**

• Whether the Users table is created correctly with proper data types

• Whether data is inserted correctly into all fields
• Whether SELECT queries return correct and expected results
• Whether UPDATE query changes the right records
• Whether DELETE query removes only the correct records
• Whether constraints like NOT NULL, UNIQUE, and ENUM are working

- **What will we NOT test?**

• The front-end or UI of the application
• API calls or server-side code
• Performance or speed of queries

- **Possible Risk**

Risk	What can go wrong	How to handle it
Low code delivery	Test window shrinks, bugs slip to production	Add time buffer; start non- functional testing early
Unclear Requirements	Tests written for wrong features, rework increases	Freez requirements before testing; use change control
Tester unavailable	Coverage drop,timeliness slip	Cross-train team; document all test cases clearly
Regression bugs	A fix breaks already-working features	Run automated regression on every build

- **Test Data Used**

• 5 users inserted with different statuses: Active, Inactive, Pending
• One user (Chandrakant) has LastLogin older than 1 year - used for DELATE testing.
• One user (Suraj) has Pending status - used for UPDATE testing
• All emails are in proper format for LIKE query testing

- **How will we validate?**

• Run SQL queries directly in phpMyAdmin after each operation
• Compare expected output with actual query result

• Take screenshots of results as proof

2. Requirements Traceability Matrix: Create a Requirements Traceability Matrix linking user requirements to database validation queries

ID No	Requirement	Test Case	SQL Query Used	Expected Result
1	System should insert user records correctly	Verify total number of users	`SELECT COUNT(*) FROM users;`	Total 5 records should be present
2	System should display active users	Check users with Active status	`SELECT * FROM users WHERE Status='Active';`	4 users should be Active
3	System should display inactive users	Check users with Inactive status	`SELECT * FROM users WHERE Status='Inactive';`	1 user should be Inactive
4	System should fetch user details using email	Search user by email	`SELECT * FROM users WHERE Email='khimali@example.com';`	Correct user details should be displayed
5	System should track recent login users	Check users with recent login date	`SELECT * FROM users WHERE LastLogin > '2026-01-01';`	Users with recent login should be displayed
6	System should not allow duplicate UserID	Check for duplicate UserIDs	`SELECT UserID, COUNT(*) FROM users GROUP BY UserID HAVING COUNT(*) > 1;`	No duplicate records should be found
7	Password field should not be empty	Check for empty passwords	`SELECT * FROM users WHERE Password='';`	No records should be returned
8	System should contain admin user	Verify admin user exists	`SELECT * FROM users WHERE Username='admin_user';`	Admin user record should be present

3. End-to-End Test Scenario: Write one End-to-End test scenario covering:

User Registration → Database Entry Validation → Profile Update → Database Update validation

Step	What we do	SQL Query	Check in DB	Expected result
1	Shubham_patil registers as a new user	INSERT INTO `users`(`User ID`, `Username`, `Email`, `Password`, `Status`, `LastLogin`) VALUES ('1', 'shubham_patil', 'shubham@example.com', 'pass123', 'Active', '2026-04-15'),	SELECT * FROM user WHERE Email = shubham@example.com	New record found with all correct values
2	Check if shubham_patil is saved in DB correctly	No separate query - verify result of Step 1	SELECT UserName, Status FROM user WHERE Email = 'shubham@example.com	UserName = Shubham_patil, Status = Active
3	Shubham_patil updates his username	UPDATE user SET UserName = 'shubham_patil_updated' WHERE Email = 'shubham@example.com	SELECT UserName FROM user WHERE Email = 'shubham@example.com	UserName = shubham_patil_updated
4	Verify that the update is saved in DB	No separate query - verify result of Step 3	SELECT * FROM user WHERE Email = 'shubham@example.com'	Updated username is visible in the database
5	Change shubham_patil sttus to inactive	UPDATE user SET Status = 'Inactive' WHERE Email = 'shubham@example.com'	SELECT Status FROM user WHERE Email = 'shubham@example.com	Status = Inactive
6	Final check - verify all data is correct	SELECT * FROM user WHERE Email = 'manish@gmail.com	All columns checked	All data is correct and consistent in the database

4. Test Summary Report: Prepare a Test Summary Report including executed queries, defects identified, and final recommendation.

- ## Overall Testing Summary

Item	Details
Total Queries Executed	10
Passed	9
Failed	1
Database	MariaDB - phpMyAdmin
Table Tested	users (assessment database)
Tester	Khimali Sonawane
Date	May 2026

- ## Queries Run and Results

Id No	Type	Query	Result	Status
1	CREATE	CREATE TABLE user (...)	Table created successfully	PASS
2	INSERT	INSERT INTO user ... x5 rows	5 users added	PASS
3	SELECT	WHERE Status = 'Active'	2 rows returned	PASS
4	SELECT	WHERE Status = 'Inactive'	2 rows returned	PASS
5	SELECT	WHERE Email LIKE '%@gmail.com	5 rows returned	PASS
6	SELECT	ORDER BY UserName ASC	Sorted correctly	PASS
7	SELECT	WHERE Status='Active' AND Email LIKE '%@gmail.com	2 rows returned	PASS

8	SELECT	WHERE LastLogin IS NOT NULL	5 rows returned	PASS
9	UPDATE	SET Status='Inactive' WHERE Status='Pending'...	1 row updated	PASS
10	DELATE	WHERE LastLogin < CURRENT_DATE - INTERVAL 1 YEAR	1 row deleted	PASS

✓ Final Recommendation

Overall, the database testing for the User Management System went well. Most queries worked correctly on the first try. The Users table was properly created with all required fields and constraints

- The table structure is correct and all constraints are working properly

• All SELECT queries returned the expected results.

• UPDATE and DELETE queries worked after adjusting the date conditions in the query.

• It is better to set default values using SQL instead of relying on the GUI.

• Test data dates should be planned carefully so they match the query conditions.

• The database is ready and can be connected to the application after fixing the minor issues mentioned above